Trabajo Final

Autómatas Finitos, de Pila y Máquina de Turing

"El As del Barrio"



Fundamentos teóricos de la informática.

Facultad de Ingeniería – UNPSJB – 2026

Grupo 4

Ayala Emiliano

Curallan Lucas

Pintado José

Vilte Elias

Índice

1. Introducción 2
2. Marco Teórico 2
3. Problemática 5
4. Composición de los autómatas 6
5. Integración y prototipo 13
6. Conclusión 14
7. Repositorio 14
8. Bibliografía 15

1. Introducción

Este trabajo busca aplicar la teoría de autómatas finitos (vista en la materia Fundamentos Teóricos de Informática) al modelado de un juego de cartas: el Blackjack (o “21”). La idea es mostrar que las reglas de un juego pueden describirse formalmente como un conjunto de estados y transiciones, y que ese modelo formal puede implementarse directamente en un programa.

Modelamos el juego con cuatro modelos complementarios, uno por cada nivel visto en la materia. Un autómata finito determinista describe el flujo de una partida (repartir, turno del jugador, turno de la casa, resultado). Un segundo autómata describe el puntaje de una mano carta por carta, incluido el doble valor del As: ese doble valor da lugar naturalmente a un autómata no determinista, que después convertimos en determinista y presentamos como máquina secuencial de Moore, que emite en cada estado una salida para informar cómo está la mano.

Los temas de los TP3 y TP4 completan el modelo: un autómata de pila lleva el balance de la sesión (cuántas manos ganó y perdió el jugador) y una máquina de Turing compara los puntajes y decide el resultado de cada mano. Los cuatro modelos se unen en una partida jugable, por consola y en una página web.

2. Marco Teórico

En esta sección definiremos los modelos teóricos a utilizar para modelar la lógica del black jack respetando la notación formal establecida por la cátedra.

2.1 Autómata Finito

Un autómata finito es un modelo matemático abstracto compuesto por una cinta de entrada finita, un cabezal de lectura y un conjunto finito de estados. El modelo lee los símbolos de la cinta uno a uno y según el estado actual sumado al símbolo leído transiciona a un nuevo estado. Si al terminar de procesar toda la cadena de entrada el sistema se encuentra en un estado de aceptación (o final), la palabra es aceptada, en caso contrario es rechazada.

2.2 Autómata Finito Determinista (AFD)

Un autómata finito es determinista si para cada combinación de estado actual y símbolo de entrada existe una única transición posible.

Formalmente un AFD se define mediante la quíntupla:

$$AFD = (\Sigma, Q, f, q_0, F)$$

Donde:

- Σ: Alfabeto finito de entrada.
- Q: Conjunto finito y no vacío de estados.
- $f: Q \times \Sigma \rightarrow Q$: Función de transición determinista.

- $q_0 \in Q$: Estado inicial.
- $F \subseteq Q$: Conjunto de estados finales o de aceptación.

2.3 Autómata Finito No Determinista (AFND)

A diferencia del AFD, un AFND permite que una misma combinación de estado y símbolo de entrada tenga múltiples estados como destinos posibles. Esta característica permite evaluar caminos en paralelo o simplificar sistemas con ambigüedad.

Siguiendo la notación de la cátedra, un AFND se define formalmente como

$$AFND = \langle Q, \Sigma, Q_0, F, f \rangle$$

Donde la función/relación de transición mapea hacia el conjunto potencia de estados

$$f: Q \times \Sigma \rightarrow 2^Q$$

y $Q_0 \subseteq Q$ representa el conjunto de estados iniciales.

2.4 Máquinas secuenciales de Estado Finito – Modelo de Moore

Una máquina secuencial extiende el modelo de autómata finito agregando la capacidad de emitir símbolos hacia un alfabeto de salida. En el Modelo de Moore, la salida emitida por la máquina depende exclusivamente del estado al que se transiciona y no de la entrada directa.

Se define formalmente mediante la séxtupla:

$$\textbf{MEF}_{\textbf{Moore}} \langle Q, \Sigma_E, \Sigma_S, f, g, q_0 \rangle$$

Donde:

Q: Conjunto finito de estados.

Σ_E: Alfabeto de entrada.

Σ_S: Alfabeto de salida.

$f: Q \times \Sigma_E \rightarrow Q$: Función de transición de estados.

$g: Q \rightarrow \Sigma_S$: Función de salida que asigna un símbolo de salida a cada estado.

$q_0 \in Q$: Estado inicial.

2.5 Gramáticas y Expresiones Regulares

Los lenguajes reconocidos por los autómatas finitos se denominan Lenguajes Regulares, Estos lenguajes pueden ser generados por una gramática regular (GR), la cual se define mediante la cuádrupla

$$\boldsymbol{G} = \langle \Sigma_N, \Sigma_T, P, S \rangle$$

Donde Σ_N es el conjunto de símbolos no terminales (asociados a los estados del autómata), Σ_T es el conjunto de terminales (alfabeto de entrada), P es el conjunto de reglas de producción y S es el axioma inicial. Así mismo , estos lenguajes pueden expresarse de forma concisa

mediante Expresiones Regulares (ER) construidas a partir de operaciones de unión (+), concatenación (.) y clausura de Kleene (*)

2.6 Gramáticas Libres de Contexto y Autómata de Pila

Los lenguajes regulares no alcanzan cuando hay que contar sin límite o recordar en espejo (por ejemplo $a^n b^n$). Esos lenguajes son los **libres de contexto** (tipo 2): los genera una GLC = < ΣN, ΣT, P, S > cuyas reglas tienen un solo no terminal a la izquierda (N → β), y los reconoce un **autómata de pila**, que es un autómata finito al que se le agrega una pila de memoria ilimitada (el último que entra es el primero que sale).

Siguiendo la notación de la cátedra, un AP se define como:

$$AP = \langle Q, \Sigma_E, \Sigma_P, q_0, z_0, F, f \rangle$$

Donde:

- Σ_P: alfabeto de la pila, y $z_0 \in \Sigma_P$ el símbolo de pila vacía (Z).
- f: (Q − F) × (Σ_E ∪ {λ, Δ}) × Σ_P → 2^(Q × Σ_P*), donde λ indica que no se lee nada y Δ el fin de la cadena.

En cada paso el AP lee un símbolo, saca el tope de la pila (pop), apila una cadena (push, que queda con su primer símbolo arriba) y cambia de estado. Si se quiere conservar el tope hay que volver a apilarlo. Una configuración se escribe (lo que falta leer, estado, pila), y el AP acepta si termina de leer la cadena en un estado final.

2.7 Máquina de Turing

La máquina de Turing es el modelo más potente: reconoce los lenguajes tipo 0 (irrestrictos) y representa cualquier algoritmo. Tiene una cinta infinita llena de blancos (□) con la palabra escrita y un cabezal que lee, escribe y se mueve hacia los dos lados.

$$MT = \langle Q, \Sigma_E, \Sigma_A, q_0, F, f \rangle$$

Donde:

- Σ_A: alfabeto auxiliar (incluye el blanco □).
- f: (Q − F) × (Σ_E ∪ Σ_A) → Q × (Σ_E ∪ Σ_A) × {I, N, D}: izquierda, no se mueve, derecha. En las tablas usamos L, S y R, como JFLAP.

En cada paso lee el símbolo, escribe otro (o el mismo), mueve el cabezal y cambia de estado. Los estados finales no tienen transiciones: la máquina acepta si llega a uno y rechaza si se bloquea. Una configuración se escribe con el estado justo antes del símbolo que se está leyendo ($\square c_1 \ldots c_{i-1}\ q_k\ c_i \ldots c_n \square$). La MT puede usarse como reconocedora o como calculadora.

2.8 Jerarquía de Chomsky

Cada tipo de gramática tiene una máquina que reconoce sus lenguajes, y cada nivel contiene a los anteriores. La tabla muestra en qué nivel está cada parte del trabajo:

Tipo	Gramática	Máquina	En el proyecto
3	Regular (GR)	AFD / AFND	Flujo de la partida y puntaje de la mano (AFND → AFD → Moore)

Tipo	Gramática	Máquina	En el proyecto
2	Libre de contexto (GLC)	Autómata de pila	Balance de la sesión
1	Dependiente del contexto (GDC)	Autómata linealmente limitado	No se usa
0	Irrestricta (GI)	Máquina de Turing	Resultado de la mano

Así, el autómata de pila aparece porque el nivel anterior no alcanza: un AFD no puede llevar la cuenta de una sesión sin límite de manos. Con la máquina de Turing el caso es distinto: comparar dos números en unario también lo puede hacer un autómata de pila, y como los puntajes no pasan de 21 alcanzaría incluso con un autómata finito. La usamos igual para mostrar el modelo más general de la materia funcionando dentro del juego, con la técnica de ir y volver tachando sobre la cinta, y como máquina que decide entre tres resultados (gana, pierde o empata).

3. Problemática

3.1. Enunciado

Blackjack es un juego de cartas en el que un jugador se enfrenta a la casa (mediante el denominado crupier). El objetivo es sumar con las cartas un valor lo más cercano posible a 21 sin pasarse, y superar a la casa. Las reglas que modelamos son:

- Se usa un mazo francés. Las cartas del 2 al 10 valen su número; J, Q y K valen 10; el As vale 1 u 11, lo que más le convenga a la mano.
- Al empezar, el jugador y la casa reciben dos cartas cada uno. La segunda carta de la casa está sin revelar.
- En su turno, el jugador puede pedir cartas cuantas veces quiera o plantarse siempre y cuando no supere 21, al hacerlo se excede y pierde.
- Si el jugador suma 21 con sus dos primeras cartas, obtiene blackjack natural y pasa al turno de la casa automáticamente.
- Cuando el jugador se planta u obtiene blackjack natural, es el turno de la casa: La misma está obligada a pedir carta mientras sume menos de 17 y a plantarse con 17 o más.
- Si la casa se pasa de 21, gana el jugador. Si no, gana quien esté más cerca de 21; con igual puntaje hay empate.
- Al terminar una mano, el jugador puede finalizar la partida o jugar otra.

3.2. Alcance y simplificaciones

Para mantener el modelo dentro del alcance de los autómatas finitos y acotarlo al tiempo del proyecto, se establecen las siguientes simplificaciones:

- Modalidad: 1 Jugador contra la casa.
- Mazo: Se asume un mazo continuo/infinito con reemplazo, por lo que la probabilidad de recibir cualquier carta es constante.
- Reglas omitidas: No se contempla la división de pares, doblar apuesta.
- Sin apuestas: El sistema modela el flujo de la partida y el cálculo de puntajes, omitiendo la gestión de fichas o dinero.

3.3. Propuesta

Para solucionar la problemática planteada y no usar un solo autómata complejo optamos por modelar el Blackjack mediante la interacción de cuatro modelos complementarios:

- Autómata de flujo de partida(AFD): Se encarga de la lógica general del juego. Modela las transiciones entre el inicio, las decisiones de los jugadores, la jugada del crupier y los estados finales de victoria, derrota o empate.
- Autómata de puntaje de la mano (AFND → AFD → MEF de Moore): Procesa la secuencia de cartas una a una para calcular el valor total. Inicialmente se define como un AFND para absorber la ambigüedad del As (que puede valer 1 u 11), obteniendo luego el AFD equivalente de 33 estados. A este último le agregamos una función de salida de Moore para alertar el estado de la mano (segura, riesgosa, alta, 21 o pasada)
- Autómata de pila del balance (AP): Lee la misma cadena de eventos que el autómata de flujo y usa la pila para contar las manos ganadas y perdidas. Acepta las sesiones que terminan sin pérdida.
- Máquina de Turing del resultado (MT): Recibe los puntajes del jugador y de la casa en unario y decide si la mano se gana, se pierde o se empata. Su resultado es el evento v, d o e que lee el autómata de flujo.

La casa reutiliza el mismo autómata de puntaje para su turno. Los cuatro modelos se implementan en Python y se unen en una partida jugable por consola y en una página web.

4. Composición de los autómatas

AUTÓMATA 1 - CONTROL DEL FLUJO(AFD)

El primer autómata maneja el flujo de la partida

$$AFDflujo = (\Sigma, Q, f, q_0, F)$$

- Σ (Alfabeto de entrada): Representa las acciones del jugador y los eventos de la partida

  Σ={r, p, s, x, h, n, v, d, e, c, t}

  - r: Repartir cartas iniciales.
  - p: Pedir carta.
  - s: Plantarse.
  - x: Jugador se pasa de 21.
  - h: La casa pide carta.
  - n: Blackjack natural(21 con las primeras dos cartas)
  - v,d,e: Resultados de victoria(jugador) derrota(gana la casa) o empate
  - c: Continuar con una nueva mano.
  - t: Terminar la partida.

- Q(Conjunto de estados) Modela las fases de la partida:
  - INICIO (q0): Estado inicial donde la mesa está lista para repartir.
  - REPARTO (q1): Se repartieron las cartas iniciales; solo desde acá puede leerse n (blackjack natural).

- TURNO_JUGADOR (q2): Fase activa donde el jugador elige pedir o plantarse
- TURNO_CASA (q3): El jugador se plantó y la casa pide cartas automáticamente.
- GANA,PIERDE,EMPATE (q4, q5, q6): Estados de resolución de la mano.
- FIN (q7): Estado final donde se concluye la partida.

- q_0 (Estado inicial): INICIO
- F(Conjunto de estados finales): FIN
- f (Función de transición): Q X Σ → Q.

Tabla De transiciones

f_F	r	n	p	s	x	h	v	d	e	c	t
→ q_0	q_1	–	–	–	–	–	–	–	–	–	–
q_1	–	q_3	q_2	q_3	q_5	–	–	–	–	–	–
q_2	–	–	q_2	q_3	q_5	–	–	–	–	–	–
q_3	–	–	–	–	–	q_3	q_4	q_5	q_6	–	–
q_4	–	–	–	–	–	–	–	–	–	q_0	q_7
q_5	–	–	–	–	–	–	–	–	–	q_0	q_7
q_6	–	–	–	–	–	–	–	–	–	q_0	q_7
* q_7	–	–	–	–	–	–	–	–	–	–	–

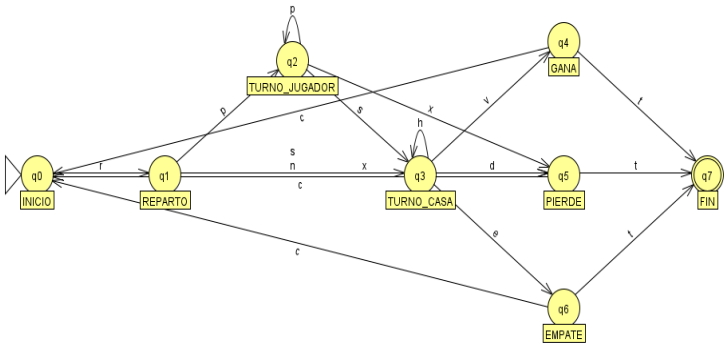
Expresión Regular (ER)

Mano individual (M):

M = r .(p*x +(n+ p*.s) . h* . (v + d + e))

Partida Completa (ER)

ER= M . (c . M)* . t



Gramática regular (GR)

Como el lenguaje del flujo es regular, también lo genera una gramática regular por la derecha. La obtenemos de la tabla de transiciones: cada estado pasa a ser un no terminal (S para q_0 y N_i para q_i), cada transición $f(q_i, a) = q_j$ da la regla $N_i \rightarrow aN_j$, y cuando q_j es el estado final q_7 la regla es $N_i \rightarrow a$.

$G_{flujo} = < \Sigma_N, \Sigma_T, P, S >, \quad \Sigma_N = \{ S, N_1, \ldots, N_6 \}, \quad \Sigma_T = \Sigma$

$S \rightarrow rN_1$
$N_1 \rightarrow nN_3 \mid pN_2 \mid sN_3 \mid xN_5$
$N_2 \rightarrow pN_2 \mid sN_3 \mid xN_5$
$N_3 \rightarrow hN_3 \mid vN_4 \mid dN_5 \mid eN_6$
$N_4 \rightarrow cS \mid t$
$N_5 \rightarrow cS \mid t$
$N_6 \rightarrow cS \mid t$

Por ejemplo, la sesión rshvt (una mano que gana el jugador) se deriva así:

$S \rightarrow rN_1 \rightarrow rsN_3 \rightarrow rshN_3 \rightarrow rshvN_4 \rightarrow rshvt$

En cambio rpst no se puede derivar: después de la s el único no terminal es N_3, que no tiene ninguna regla con t (antes hay que pasar por v, d o e). Es la misma cadena que el AFD rechaza por bloqueo en q_3.

AUTÓMATA 2 - PUNTAJE (AFND→ AFD→ MEF)

Cada carta es un símbolo de entrada. El 10, la J, la Q y la K valen lo mismo y se agrupan en T; el As vale 1 u 11:

$$\Sigma_E = \{ A, 2, 3, 4, 5, 6, 7, 8, 9, T \}$$

AFND

El estado p_i representa una interpretación de la mano que suma i. Ante una carta distinta del As la interpretación avanza $p_{i+v(c)}$; ante un As se divide en dos (As= 1 y As = 11) las interpretaciones que superan 21 no tienen transición.

$$AFND_P = < Q_N, \Sigma_E, Q_0, F_N, f_N >, \quad Q_N = \{ p_0, ..., p_{21} \}, \quad Q_0 = \{ p_0 \}, \quad F_N = \{ p_{21} \}$$

$$f_N(p_i, c) = \{ p_{i+v(c)} \} \text{ si } i + v(c) \leq 21 \ (\emptyset \text{ si no}), \quad \text{para } c \neq A$$
$$f_N(p_i, A) = \{ p_j / j \in \{ i+1, i+11 \}, j \leq 21 \}$$

Reconoce las manos que pueden valer exactamente 21. por ejemplo con A55 hay dos caminos: con As= 1 termina en p_{11} y con As=11 en p_{21}; como uno llega a un estado final, la palabra se acepta.

AFD equivalente

Para el juego hace falta un AFD: cada estado del AFD representa un conjunto de estados en que podría estar el AFND tras leer las mismas cartas. como a lo sumo un As puede valer 11 sin pasarse, cada conjunto alcanzable tiene a lo sumo dos elementos y se nombran con la terminología del juego:

- $D_k = \{ p_k \}$: mano dura de valor k (D_0, D_2, …, D_{21}: 21 estados).

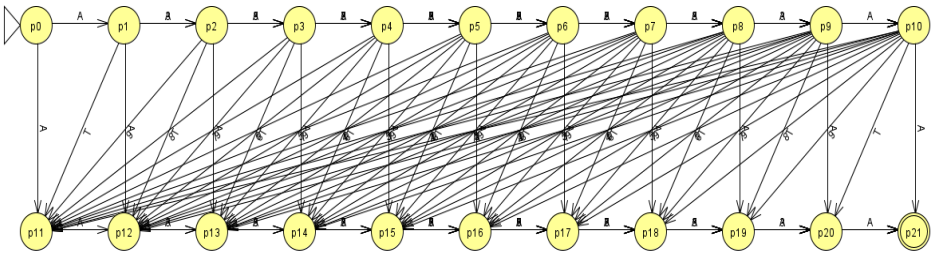
- $B_k = \{ p_{k-10}, p_k \}$: mano blanda, un As vale 11 (B_{11}, …, B_{21}: 11 estados).
- X = ∅: mano pasada, estado sumidero.

$$AFD_P = (\Sigma_E, Q_D, f_D, D_0, F_D), \qquad |Q_D| = 33, \qquad F_D = \{ D_{21}, B_{21} \}$$

En lugar de una tabla de 33 × 10, f_D se resume en reglas (v = valor de la carta c ≠ A):

Estado actual	Lee A	Lee c ≠ A
D_k, k ≤ 10	B_{k+11}	D_{k+v}
D_k, k ≥ 11	D_{k+1} si k < 21; X si k = 21	D_{k+v} si k + v ≤ 21; X si no
B_k	B_{k+1} si k < 21; D_{12} si k = 21	B_{k+v} si k + v ≤ 21; D_{k+v-10} si no
X	X	X

Cuando contar el As como 11 pasaría la mano, el As pasa a valer 1 y la mano se vuelve dura (B_{17} con 5 → D_{12}). Por minimización, como reconocedor solo D_{10} y B_{20} son equivalentes (AFD mínimo de 32 estados); con la salida de Moore se distinguen y los 33 estados son necesarios.



Máquina secuencial de Moore

Al AFD le agregamos una función de salida g : $Q_D \rightarrow \Sigma_S$ que informa el estado de la mano después de cada carta, según su mejor valor v (el mayor que no supera 21):

$$MEF_P = < Q_D, \Sigma_E, \Sigma_S, f_D, g, D_0 >, \quad \Sigma_S = \{ S, R, A, V, P \}$$

g	Significado	Estados	Uso
S	Segura: ninguna carta la pasa	D_0, D_2…D_{11}, B_{11}…B_{16}	La casa pide
R	Riesgosa: dura entre 12 y 16	D_{12}…D_{16}	La casa pide
A	Alta: entre 17 y 20	D_{17}…D_{20}, B_{17}…B_{20}	La casa se planta
V	Veintiuno	D_{21}, B_{21}	Natural si son 2 cartas
P	Pasada	X	Pierde quien la tiene

Así, la regla de la casa queda expresada directamente con la salida: pide si g ∈ {S, R} y se planta si g ∈ {A, V}. Como el 17 blando (B_{17}) tiene salida A, la regla del 17 blando ya está incluida. Por ejemplo, para la mano A65 la máquina emite S A R: el As con el 6 forma un 17 blando, y con el 5 el As pasa a valer 1 y queda un 12 duro.

AUTÓMATA 3 - BALANCE DE LA SESIÓN (AP)

El autómata de flujo reconoce si una sesión es válida, pero no lleva la cuenta de cuántas manos ganó o perdió el jugador. Queremos reconocer las sesiones que terminan **sin pérdida**:

$$L_B = \{\, w \in L(AFD_{flujo}) \;/\; \#v(w) \geq \#d(w) + \#x(w) \,\}$$

L_B no es regular: con c se pueden jugar manos sin límite y la diferencia entre ganadas y perdidas puede ser cualquier número, mientras que un AFD solo tiene una cantidad fija de estados para recordarla. Usamos la técnica del ejercicio 14 del TP3: la pila guarda lo que sobra, y una mano del signo contrario se cancela con un pop.

$$AP_B = \langle Q, \Sigma, \Sigma_P, q_0, Z, F, f \rangle$$

$$Q = \{\, q_0, \ldots, q_8 \,\},\;\; \Sigma_P = \{\, Z, G, P \,\},\;\; F = \{\, q_8 \,\}$$

- Σ es el mismo alfabeto del autómata de flujo y $q_0 \ldots q_7$ son sus mismos estados (q_7 FIN deja de ser final).
- q_8 (SIN_PERDIDA) es el nuevo estado final, al que se llega leyendo Δ desde q_7.
- G representa una mano ganada de más y P una perdida de más.

Función de transición

La función copia la del AFD de flujo (mismo estado destino) y solo cambia lo que se apila según el resultado de la mano:

Lee	Tope Z	Tope G	Tope P
v (q4)	push GZ	push GG	push λ (se cancela)
d (q5), x (q5)	push PZ	push λ (se cancela)	push PP
resto de los símbolos	push Z	push G	push P
Δ en q7	q8, push Z	q8, push G	— (bloqueo)

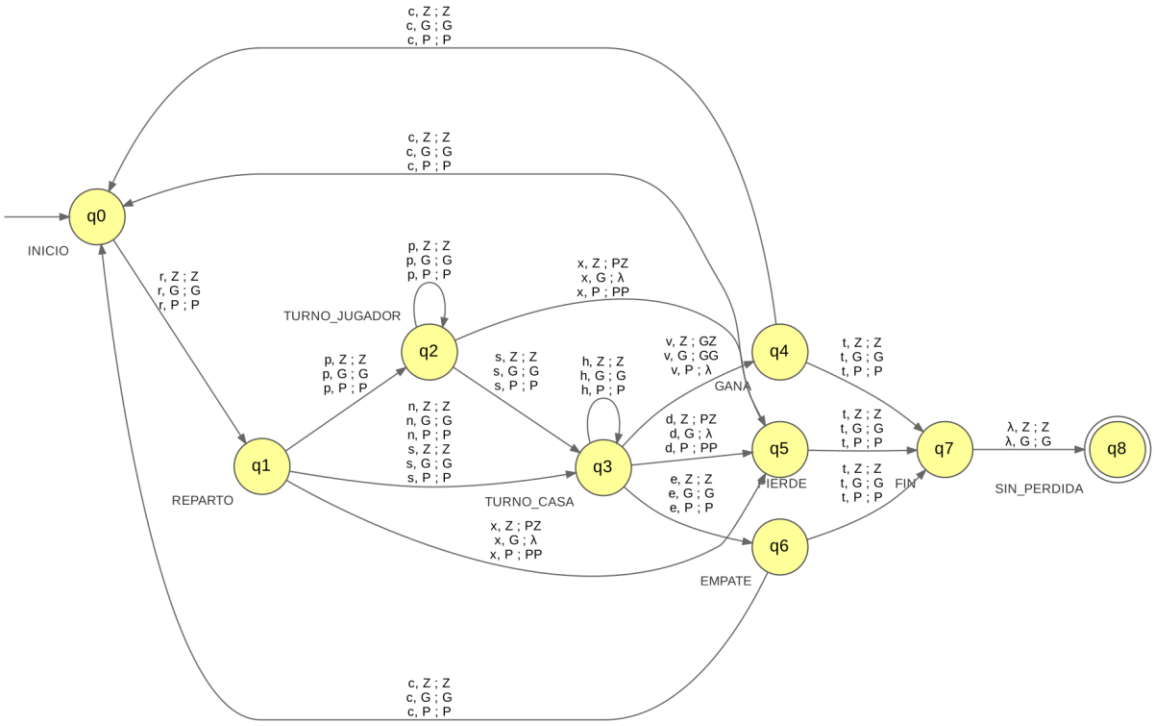
Para cada (estado, símbolo, tope) hay una sola transición, así que el AP es **determinista** (56 transiciones en total). Además es un ejemplo de una propiedad vista en el TP3: la intersección de un lenguaje regular con uno libre de contexto es libre de contexto. Los estados controlan la partida y la pila lleva el balance.

Ejemplos de aceptación

rxcrshvt (pierde la primera mano y gana la segunda) es aceptada:

`(rxcrshvtΔ, q0, Z) ⊢ (xcrshvtΔ, q1, Z) ⊢ (crshvtΔ, q5, PZ) ⊢ (rshvtΔ, q0, PZ) ⊢ (shvtΔ, q1, PZ) ⊢ (hvtΔ, q3, PZ) ⊢ (vtΔ, q3, PZ) ⊢ (tΔ, q4, Z) ⊢ (Δ, q7, Z) ⊢ (λ, q8, Z)`

rxcrxcrshvt (pierde dos y gana una) es rechazada: al llegar a q_7 la pila queda PZ y no hay transición (q_7, Δ, P).



En JFLAP no existe el símbolo Δ, por eso en balance_pila.jff la última transición desde q_7 es una transición λ (q_7 no tiene otras, así que no se pierde el determinismo en la práctica).

AUTÓMATA 4 - RESULTADO DE LA MANO (MT)

Cuando terminan los dos turnos hay que comparar los puntajes y mandarle al flujo el evento v, d o e. Lo resolvemos con una MT reconocedora que usa la técnica de **tachar y emparejar** (TP4, ejercicio 2a). En la cinta van los dos puntajes en unario separados por #; por ejemplo, 20 contra 18 es $1^{20}\#1^{18}$.

$$MT_R = \langle Q, \Sigma_E, \Sigma_A, q_0, F, f \rangle$$

$$Q = \{ q_0, \ldots, q_7 \},\ \ \Sigma_E = \{ 1, \# \},\ \ \Sigma_A = \{ X, Y, \square \},\ \ F = \{ q_5, q_6, q_7 \}$$

- TACHA_J (q_0): tacha un 1 del jugador con X.
- BUSCA_# (q_1): avanza hasta el separador.
- TACHA_C (q_2): saltea las Y y tacha con Y la pareja de la casa.
- VUELVE (q_3): regresa hasta la última X y empieza otra vuelta.
- REVISA_C (q_4): el jugador ya no tiene 1; revisa si a la casa le sobran.
- GANA, PIERDE, EMPATE (q_5, q_6, q_7): estados de parada, sin transiciones.

Tabla de transiciones

Est. act.	Lee	Est. nue.	Escribe	Mov.
→q0	1	q1	X	R
q0	#	q4	#	R
q1	1	q1	1	R
q1	#	q2	#	R

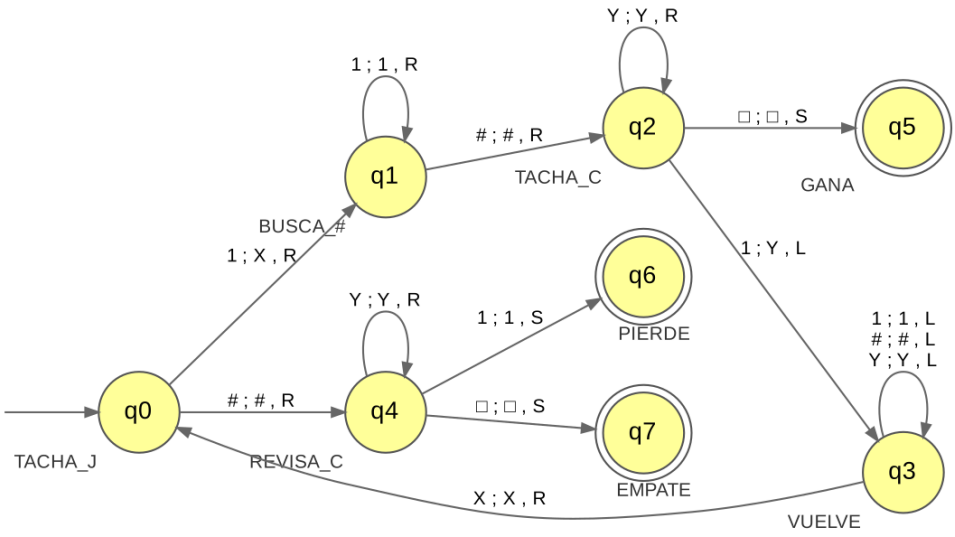
Est. act.	Lee	Est. nue.	Escribe	Mov.
q2	Y	q2	Y	R
q2	1	q3	Y	L
q2	□	q5	□	S
q3	1	q3	1	L
q3	#	q3	#	L
q3	Y	q3	Y	L
q3	X	q0	X	R
q4	Y	q4	Y	R
q4	1	q6	1	S
q4	□	q7	□	S

Si a un 1 del jugador le falta pareja termina en GANA (v); si a la casa le sobran 1 termina en PIERDE (d), y si se terminan los dos juntos en EMPATE (e). Los casos en que alguien se pasa de 21 no llegan a la MT: si se pasa el jugador el flujo ya recibió x, y si se pasa la casa la salida P del Moore da v directamente. La comprobamos para todos los pares de puntajes de 0 a 21.

Ejemplo

2 contra 1 (11#1) termina en GANA:

□q011#1□ ⊢ □Xq11#1□ ⊢ □X1q1#1□ ⊢ □X1#q21□ ⊢ □X1q3#Y□ ⊢ □Xq31#Y□ ⊢ □q3X1#Y□ ⊢ □Xq01#Y□ ⊢ □XXq1#Y□ ⊢ □XX#q2Y□ ⊢ □XX#Yq2□ ⊢ □XX#Yq5□



5. Integración y prototipo

Los cuatro modelos trabajan juntos en una partida. Cada mano del jugador y de la casa recorre el AFD de puntaje carta por carta, y los eventos que lee el autómata de flujo salen de la salida g del Moore y de la máquina de Turing:

- **r**: se reparten dos cartas a cada uno. Si la salida del jugador es V, se emite **n** (blackjack natural).
- **p** / **x**: el jugador pide carta; si la salida pasa a P (estado X) se emite x, si no p. Con 21 se planta solo.
- **s**: el jugador se planta.
- **h**: la casa pide mientras su salida sea S o R.
- **v** / **d** / **e**: si la casa terminó en P se emite v; si no, la MT compara los dos puntajes.
- **c** / **t**: el jugador elige jugar otra mano o terminar.

Después de cada mano se muestra el balance que guarda la pila del AP, y al terminar la sesión se informa si el autómata de flujo y el AP aceptan la cadena de eventos.

5.1 Versión web

La partida se puede jugar abriendo web/index.html en el navegador. Las tablas de los cuatro autómatas no se copian a mano: el script exportar_web.py las genera en web/automatas.js a partir del código Python, así la página y el programa usan las mismas funciones de transición. Los botones Repartir, Pedir y Plantarse se habilitan solo si el estado actual del flujo tiene transición con ese símbolo, y las teclas r, p, s, c y t son los símbolos del alfabeto. Un panel muestra en vivo el estado del flujo, las salidas del Moore, la cinta de la MT mientras compara y la pila del AP.



5.2 Pruebas

Las pruebas están en pruebas.py y usan semillas fijas, así que se pueden repetir. El autómata de puntaje se comparó con el cálculo directo del puntaje en 200 000 manos al azar; la minimización confirmó los 33 estados y el mínimo de 32; la MT se corrió para los 484 pares de puntajes de 0 a 21, y la partida por consola se jugó en 3000 sesiones simuladas (8536 manos, con respuestas inválidas incluidas). En todos los casos el flujo aceptó la sesión, el AP coincidió con el conteo de ganadas y perdidas y no hubo errores. La versión web se probó además con más de 160 manos automáticas en el navegador.

6. Conclusión

El trabajo nos permitió comprobar que las reglas de un juego conocido se pueden escribir como modelos formales, y que esos modelos se pueden llevar a un programa casi sin cambios: las tablas de transición del informe son las mismas que usan el código en Python y la página web.

También vimos en un caso concreto por qué existen distintos niveles en la jerarquía de Chomsky. El flujo de la partida y el puntaje de una mano son regulares; el doble valor del As se resolvió con un AFND y su AFD equivalente, y la salida de Moore alcanzó para expresar la regla de la casa. El balance de la sesión, en cambio, no se puede llevar con una cantidad fija de estados y necesitó la pila. La máquina de Turing no era indispensable para comparar dos puntajes, pero nos sirvió para aplicar la técnica de tachar y emparejar y para ver el modelo más general trabajando junto a los otros tres.

Como trabajo futuro quedan las reglas que dejamos fuera del alcance (dividir pares, doblar, apuestas) y un mazo finito sin reemplazo, que obligaría a recordar las cartas que ya salieron.

7. Repositorio

El código del prototipo y los archivos de JFLAP se encuentran en el repositorio: https://github.com/ayalaEmiliano/el-as-del-barrio

- flujo.py, puntaje.py, pila.py y turing.py: los cuatro modelos.
- partida.py: la partida por consola; main.py: ejemplos de cada modelo; pruebas.py: pruebas automáticas.
- web/: la versión web; exportar_web.py genera web/automatas.js.
- jflap/: los autómatas en JFLAP y sus grafos.

8. Bibliografía

- Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Pearson Educación.
- Cátedra de Fundamentos Teóricos de la Informática, UNPSJB (2026). Apuntes: *Autómatas Finitos (AFD)*, *Autómatas Finitos (AFND)*, *Máquinas Secuenciales*, *Expresiones Regulares*, *Lenguajes Regulares*, *Gramáticas Libres del Contexto*, *Autómata de Pila*, *Máquina de Turing*.
- Cátedra de Fundamentos Teóricos de la Informática, UNPSJB (2026). Trabajos Prácticos 3 (Lenguajes Libres de Contexto, Autómatas a Pila, Lenguajes Sensibles al Contexto) y 4 (Máquinas de Turing).
- Rodger, S. H. y Finley, T. W. (2006). *JFLAP: An Interactive Formal Languages and Automata Package*. Jones & Bartlett. https://www.jflap.org/
- automathon (librería de Python). https://pypi.org/project/automathon/